

```
!nvidia-smi
```

```
Tue Apr 21 12:29:57 2026
```

```
+-----+
| NVIDIA-SMI 580.82.07                  Driver Version: 580.82.07          CUDA Version: 13.0     |
+-----+-----+-----+-----+-----+
| GPU   Name                               Persistence-M   Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf              Pwr:Usage/Cap     Memory-Usage | GPU-Util  Compute M. |
|                               MIG M.           |                      |
+=====+=====+=====+=====+=====+
|   0   Tesla T4                          Off          00000000:00:04.0 Off |             0        |
| N/A   44C    P8              10W /   70W      3MiB / 15360MiB |      0%      Default  |
|                               N/A             |                      |
+-----+-----+-----+-----+-----+

+-----+
| Processes:                                |
| GPU   GI    CI          PID    Type    Process name                  GPU Memory |
|          ID    ID                                   Usage          |
+=====+
| No running processes found              |
+-----+
```

```
import kagglehub
```

```
# Download latest version
```

```
path = kagglehub.dataset_download("puneet6060/intel-image-classification")
```

```
print("Path to dataset files:", path)
```

```
Using Colab cache for faster access to the 'intel-image-classification' dataset.
Path to dataset files: /kaggle/input/intel-image-classification
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
from torchvision import datasets, transforms, models
```

```

from torch.utils.data import DataLoader
from tqdm import tqdm
import matplotlib.pyplot as plt

# ===== DEVICE =====
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# ===== CONFIG =====
DATA_DIR = "/kaggle/input/intel-image-classification"
TRAIN_DIR = f"{DATA_DIR}/seg_train/seg_train" # Corrected path
VAL_DIR = f"{DATA_DIR}/seg_test/seg_test" # Corrected path

BATCH_SIZE = 32
EPOCHS = 15
LR = 1e-4

MODEL_LIST = ["alexnet", "vgg16", "vgg19"]

# ===== TRANSFORMS =====
train_tf = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomResizedCrop(224, scale=(0.7, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(0.2, 0.2, 0.2),
    transforms.ToTensor(),
    transforms.Normalize([0.5]*3, [0.5]*3)
])

val_tf = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize([0.5]*3, [0.5]*3)
])

# ===== DATA =====
train_data = datasets.ImageFolder(TRAIN_DIR, transform=train_tf)
val_data = datasets.ImageFolder(VAL_DIR, transform=val_tf)

```

```
val_data = datasets.ImageFolder(val_dir, transform=val_tf)

train_loader = DataLoader(train_data, batch_size=BATCH_SIZE, shuffle=True)
val_loader = DataLoader(val_data, batch_size=BATCH_SIZE)

NUM_CLASSES = len(train_data.classes)
print("Classes:", train_data.classes)

# ===== MODEL FUNCTION =====
def get_model(name):
    if name == "alexnet":
        model = models.alexnet(pretrained=True)
    elif name == "vgg16":
        model = models.vgg16_bn(pretrained=True)
    elif name == "vgg19":
        model = models.vgg19_bn(pretrained=True)
    else:
        raise ValueError("Invalid model")

    # Freeze feature layers
    for p in model.features.parameters():
        p.requires_grad = False

    # Replace classifier
    model.classifier[6] = nn.Sequential(
        nn.Dropout(0.5),
        nn.Linear(4096, NUM_CLASSES)
    )

    return model.to(device)

# ===== EVALUATION =====
def evaluate(model):
    model.eval()
    correct, total = 0, 0

    with torch.no_grad():
        for x, y in val_loader:
```

```
x, y = x.to(device), y.to(device)

out = model(x)
pred = out.argmax(1)

correct += (pred == y).sum().item()
total += y.size(0)

return 100 * correct / total

# ===== TRAIN FUNCTION =====
def train_model(model_name):
    print("\nTraining:", model_name)

    model = get_model(model_name)

    criterion = nn.CrossEntropyLoss()

    optimizer = optim.Adam(
        filter(lambda p: p.requires_grad, model.parameters()),
        lr=LR,
        weight_decay=1e-4
    )

    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
        optimizer, mode='max', patience=2, factor=0.3
    )

    best_acc = 0
    patience = 4
    counter = 0

    train_losses = []
    val_accs = []

    for epoch in range(EPOCHS):
        model.train()
        loss_sum = 0
```

```
for x, y in tqdm(train_loader):
    x, y = x.to(device), y.to(device)

    optimizer.zero_grad()
    out = model(x)

    loss = criterion(out, y)
    loss.backward()
    optimizer.step()

    loss_sum += loss.item()

avg_loss = loss_sum / len(train_loader)
acc = evaluate(model)

train_losses.append(avg_loss)
val_accs.append(acc)

print(f"Epoch {epoch+1}/{EPOCHS} | Loss: {avg_loss:.4f} | Val Acc: {

scheduler.step(acc)

# Early stopping
if acc > best_acc:
    best_acc = acc
    counter = 0
    torch.save(model.state_dict(), f"{model_name}_best.pth")
else:
    counter += 1

if counter >= patience:
    print("Early stopping triggered")
    break

return train_losses, val_accs, best_acc
```

```
# ===== RUN ALL MODELS =====
```

```
results = {}

for model_name in MODEL_LIST:
    losses, accs, best_acc = train_model(model_name)
    results[model_name] = (losses, accs, best_acc)

# ===== PLOT =====
plt.figure()

for model_name in MODEL_LIST:
    plt.plot(results[model_name][1], label=model_name)

plt.title("Validation Accuracy Comparison (Intel Dataset)")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

# ===== FINAL RESULTS =====
print("\nFinal Results:")
for model_name in MODEL_LIST:
    print(f"{model_name}: {results[model_name][2]:.2f}%")
```

```
warnings.warn(msg)
100%|██████████| 439/439 [03:14<00:00, 2.26it/s]
Epoch 1/15 | Loss: 0.4707 | Val Acc: 90.87%
100%|██████████| 439/439 [03:11<00:00, 2.30it/s]
Epoch 2/15 | Loss: 0.3298 | Val Acc: 90.83%
100%|██████████| 439/439 [03:10<00:00, 2.30it/s]
Epoch 3/15 | Loss: 0.3015 | Val Acc: 90.80%
100%|██████████| 439/439 [03:09<00:00, 2.32it/s]
Epoch 4/15 | Loss: 0.2757 | Val Acc: 91.57%
100%|██████████| 439/439 [03:12<00:00, 2.28it/s]
Epoch 5/15 | Loss: 0.2610 | Val Acc: 91.07%
100%|██████████| 439/439 [03:09<00:00, 2.31it/s]
Epoch 6/15 | Loss: 0.2493 | Val Acc: 91.77%
100%|██████████| 439/439 [03:10<00:00, 2.31it/s]
Epoch 7/15 | Loss: 0.2449 | Val Acc: 91.93%
100%|██████████| 439/439 [03:10<00:00, 2.31it/s]
Epoch 8/15 | Loss: 0.2346 | Val Acc: 91.77%
```

```
import os

print(f"Contents of the dataset root directory ({path}):")
for item in os.listdir(path):
    print(f" - {item}")

train_root = os.path.join(path, 'seg_train')
test_root = os.path.join(path, 'seg_test')

print(f"\nContents of the training data directory ({train_root}):")
if os.path.exists(train_root):
    for item in os.listdir(train_root):
        print(f" - {item}")
else:
    print("Training directory not found.")

print(f"\nContents of the validation data directory ({test_root}):")
if os.path.exists(test_root):
    for item in os.listdir(test_root):
        print(f" - {item}")
else:
    print("Validation directory not found.")
```

Contents of the dataset root directory (/kaggle/input/intel-image-classification):

- seg_train
- seg_pred
- seg_test

Contents of the training data directory (/kaggle/input/intel-image-classification/seg_train):

- seg_train

Contents of the validation data directory (/kaggle/input/intel-image-classification/seg_test):

- seg_test

